



Universidad Nacional del Comahue
Facultad de Informática
Departamento Ingeniería en Sistemas



Lenguajes de consulta relacionales comerciales

Gestión de Bases de Datos

Licenciatura en Ciencias de la Computación
3° Año

Integrantes del Grupo:

- Agustin Ferreyra – Legajo: 4025
- Milagros Ruiz – Legajo: 4564

Docentes:

- Prof. Titular: Agustina Buccella
- Ayudante: Ignacio San Pedro Delgado

Fecha de entrega:

26/10/2025

Lenguajes de consulta relacionales comerciales	1
Unidad I	3
Dominio	3
MER	4
Derivación a tablas	4
Integridad referencial	5
Dominios	6
Tuplas	7
Operaciones	11
Borrado	11
Error de actualización	13
Cambio de requerimientos del MER	13
Afirmaciones	15
Vistas	16
Unidad V - Seguridad	18
Creación de roles	18
Creacion de usuarios	19
Tabla de permisos de usuarios	21
Unidad V - Recuperación	22
Transacciones con fallo	22
Tabla 2 : Tabla de transacciones con fallo	23
Mecanismo de recuperación inmediato	23
Mecanismo de recuperación diferido	24
Mecanismo de recuperación inmediato con checkpoint	25
Mecanismo de recuperación diferido con checkpoint	26
Participaciones	27
Grupal	27
Ruiz Milagros	27
Ferreyra Agustin	27

Unidad I

1) Realice un MER y su derivación considerando que posea (al menos):

- a) una entidad débil relacionada con una relación de M.M con otra entidad
- b) una generalización/especialización en donde una generalización se relacione 1.N con otra entidad
- c) un atributo multivaluado
- d) un atributo derivado
- e) Una unión en donde una de sus subuniones se relacionen M.M con otra entidad.
- f) una relación con atributos descriptivos

Dominio

Se desea modelar una base de datos de un aeropuerto que se utiliza para realizar el seguimiento de los aviones, sus propietarios y los empleados. Para ello se recopiló la siguiente información: Cada avión tiene un número de registro único, una capacidad, un peso en toneladas y se almacena en un hangar especial. Cada hangar tiene un número, una capacidad y una ubicación, es decir que un hangar puede albergar a varios aviones. En la base de datos también se mantiene el registro de los propietarios de los aeroplanos, los mecánicos que lo han mantenido al mismo y los pilotos del aeropuerto. Los servicios de mantenimiento de un avión son realizados por mecánicos. Un avión recibe periódicamente servicios de mantenimiento que se identifican con la fecha en que son realizados al mismo, las horas dedicadas a tal obra y el tipo de trabajo realizado. Un propietario puede ser una persona física o una corporación. De los propietarios se desea registrar nombre, dirección y teléfonos. En caso de ser una persona su número de documento. Las corporaciones se identifican por su nombre único y las personas por su número de documento. De los empleados se desea registrar nombre, apellido, dirección, teléfono y la obra social que tienen registrada. De los pilotos, además de sus datos personales se registra su número de licencia. De cada obra social se conoce su nombre, su CUIT y la sede. Las corporaciones se identifican por su nombre único y las personas por su número de documento. De los empleados, además de sus datos personales se registra el monto de su salario y turno en el que trabaja. Cada persona física puede alquilar un espacio en un hangar en una fecha determinada por día.

MER

Modelo entidad relación de nuestro dominio

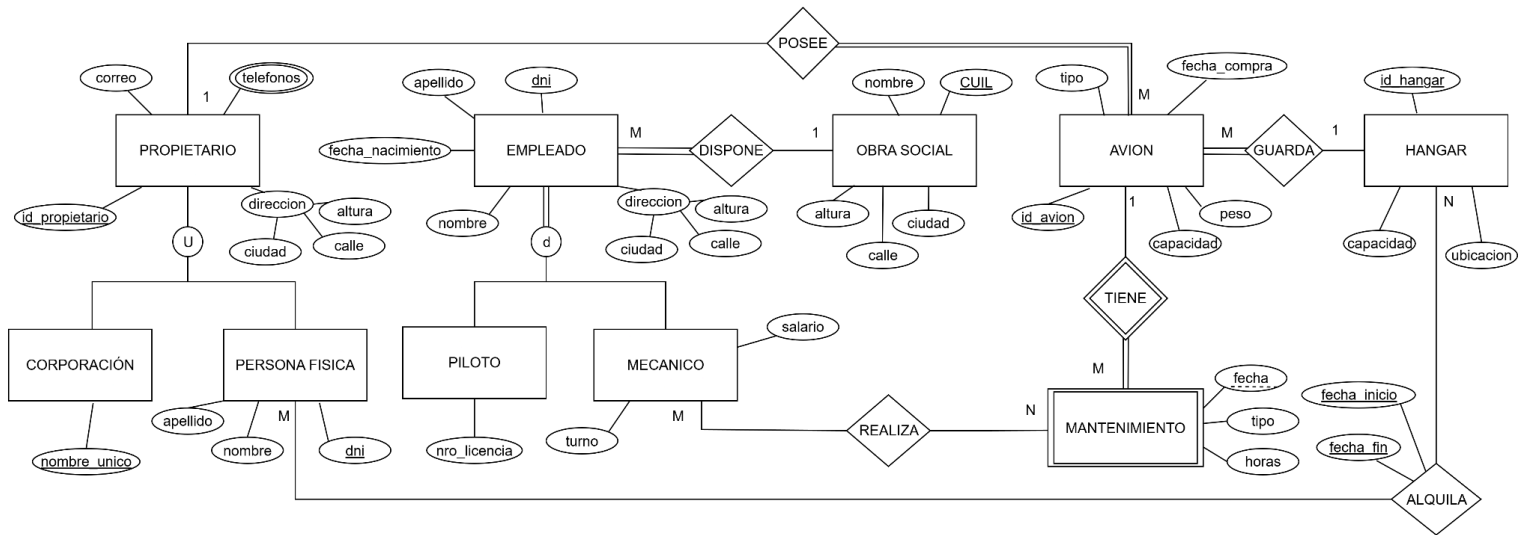


Figura 1

Derivación a tablas

Las claves primarias están indicadas con subrayado negro

Las claves foráneas están indicadas con **color rojo**

Mecanico(dni, nombre, apellido, altura, ciudad, calle, salario, turno, fecha_nacimiento, **CUIL**)

Piloto(dni, nombre, apellido, altura, ciudad, calle, nro_licencia, fecha_nacimiento, **CUIL**)

Propietario(id_propietario, correo, ciudad, altura, calle)

PersonaFisica(dni, nombre, apellido, **id_propietario**)

Corporación(nombre_único, **id_propietario**)

ObraSocial(cuil, nombre, ciudad, calle, altura)

Avion(id_avion, tipo, capacidad, peso, fecha_compra, **id_hangar**, **id_propietario**)

Hangar(id_hangar, capacidad, ubicación)

Mantenimiento(fecha, **id_avion**, tipo, horas)

Realiza(dni, **fecha**, **id_avion**)

Alquila(**id_hangar**, **dni**, fecha_inicio, fecha_fin)

Teléfonos(**id_propietario**, nro_telefono)

SQL CREACION TABLAS

```
CREATE TABLE Propietario (  
    id_propietario SERIAL PRIMARY KEY,  
    correo VARCHAR(50),  
    ciudad VARCHAR(30),  
    altura SMALLINT,  
    calle VARCHAR(30)  
);  
CREATE TABLE PersonaFisica (  
    dni domDni PRIMARY KEY,  
    nombre VARCHAR(30),  
    apellido VARCHAR(30),  
    id_propietario INT,  
    FOREIGN KEY (id_propietario) REFERENCES Propietario(id_propietario)  
  
);  
  
CREATE TABLE ObraSocial (  
    cuil domCuil PRIMARY KEY,  
    nombre VARCHAR(40),  
    ciudad VARCHAR(30),  
    calle VARCHAR(30),  
    altura SMALLINT  
);  
  
CREATE TABLE Mecánico (  
    dni domDni PRIMARY KEY,  
    nombre VARCHAR(30),  
    apellido VARCHAR(30),  
    altura SMALLINT,  
    ciudad VARCHAR(30),  
    calle VARCHAR(30),  
    salario INTEGER,  
    turno VARCHAR(15),  
    fecha_nacimiento domFechaNacimiento,  
    cuil domicilio,  
    FOREIGN KEY (cuil) REFERENCES ObraSocial(cuil)  
    ON UPDATE CASCADE ON DELETE SET DEFAULT  
);
```

```
CREATE TABLE Hangar (  

```

```

    id_hangar SERIAL PRIMARY KEY,
    capacidad SMALLINT,
    ubicacion VARCHAR(50)
);
CREATE TABLE Avion (
    id_avion SERIAL PRIMARY KEY,
    tipo VARCHAR(20),
    capacidad SMALLINT,
    peso SMALLINT,
    fecha_compra DATE,
    id_hangar INTEGER,
    id_propietario INTEGER,
    FOREIGN KEY (id_propietario) REFERENCES Propietario(id_propietario)
    ON UPDATE CASCADE ON DELETE RESTRICT
);

```

```

CREATE TABLE Piloto (
    dni domDni PRIMARY KEY,
    nombre VARCHAR(30),
    apellido VARCHAR(30),
    altura SMALLINT,
    ciudad VARCHAR(30),
    calle VARCHAR(30),
    nro_licencia VARCHAR(20),
    fecha_nacimiento domFechaNacimiento,
    cuil domCuil,
    FOREIGN KEY (cuil) REFERENCES ObraSocial(cuil)
    ON UPDATE CASCADE ON DELETE SET DEFAULT
);

```

```

CREATE TABLE Mantenimiento (
    fecha DATE,
    id_avion INT,
    tipo domTipo,
    horas SMALLINT,
    PRIMARY KEY (fecha, id_avion),
    FOREIGN KEY (id_avion) REFERENCES Avion(id_avion)
    ON UPDATE CASCADE ON DELETE CASCADE
);

```

```

CREATE TABLE Realiza (
    dni INT,
    fecha DATE,
    id_avion INT,

```

```

PRIMARY KEY (dni, fecha, id_avion),
FOREIGN KEY (dni) REFERENCES Mecanico(dni)
    ON UPDATE CASCADE ON DELETE SET NULL,
FOREIGN KEY (fecha, id_avion) REFERENCES Mantenimiento(fecha, id_avion)
    ON UPDATE CASCADE ON DELETE CASCADE
);

```

```

CREATE TABLE Alquiler (
    id_hangar INT,
    dni INT,
    fecha_inicio DATE,
    fecha_fin DATE,
    PRIMARY KEY (id_hangar, dni, fecha_inicio, fecha_fin),
    FOREIGN KEY (id_hangar) REFERENCES Hangar(id_hangar)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    FOREIGN KEY (dni) REFERENCES PersonaFisica(dni)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

```

```

CREATE TABLE Telefonos (
    id_propietario INT,
    nro_telefono VARCHAR(15),
    PRIMARY KEY (id_propietario, nro_telefono),
    FOREIGN KEY (id_propietario) REFERENCES Propietario(id_propietario)
        ON UPDATE CASCADE ON DELETE CASCADE
);

```

```

CREATE TABLE Corporacion (
    nombre_unico VARCHAR(40) PRIMARY KEY,
    id_propietario INT,
    FOREIGN KEY (id_propietario) REFERENCES Propietario(id_propietario)
        ON UPDATE CASCADE ON DELETE CASCADE
);

```

2) Realizar el DDL del MER anterior completo considerando:

- a) Integridad referencial donde exista al menos un CASCADE, SET NULL, RESTRICT y un SET DEFAULT

Integridad referencial

A continuación se detalla cada una de las políticas de actualización y borrado seleccionadas para cada relación y una justificación de la opción elegida

- **Avión:** tiene como clave foránea id_propietario y la política para actualización es cascade ya que si queremos modificar el id en la tabla propietario debemos hacerlo en la tabla avion para preservar la integridad, y para borrado tenemos la política de restrict ya que no debemos permitir eliminar un propietario si aun posee aviones registrados en nuestra base de datos.
- **Corporación:** tiene como clave foránea id_propietario y la política para actualización y borrado es cascade ya que tanto como si queremos modificar o eliminar el id en la tabla propietario este cambio tendría que replicarse en la tabla corporación, no nos interesa mantener una corporación que no es propietaria.
- **Mantenimiento:** tiene como clave foránea id_avion y la política para actualización y borrado es cascade ya que tanto como si queremos modificar o eliminar un avion este cambio tendría que replicarse en la tabla mantenimiento, no nos interesa mantener los mantenimientos realizados a aviones que no pertenecen a nuestra base de datos.
- **Mecánico:** tiene como clave foránea cuil y la política para actualización es cascade ya que si queremos modificar el cuil en la tabla obra social debemos hacerlo en la tabla mecánico para preservar la integridad, y para borrado tenemos la política de set default ya que un empleado puede dejar de tener obra social y en ese caso se asigna un número por defecto (1111111111).
- **Persona Física:** tiene como clave foránea id_propietario y la política para actualización es cascade ya que si queremos modificar el id en la tabla propietario este cambio tendría que replicarse en la tabla persona fisica, para borrado tenemos la política de set null ya que una persona física puede no ser dueño pero aun asi alquilar un hangar por lo que no se requiere borrar la tupla en caso de borrar al propietario.
- **Piloto:** tiene como clave foránea cuil y la política para actualización es cascade ya que si queremos modificar el cuil en la tabla obra social debemos hacerlo en la tabla piloto para preservar la integridad, y para borrado tenemos la política de set default ya que un empleado puede dejar de tener obra social y en ese caso se asigna un número por defecto (1111111111).
- **Teléfonos:** tiene como clave foránea id_propietario y la política para actualización y borrado es cascade ya que tanto como si queremos modificar o eliminar el id en la tabla propietario

este cambio tendría que replicarse en la tabla telefono, no nos interesa mantener telefonos de alguien que no se encuentra en nuestra base de datos.

- **Alquila:** tiene como claves foraneas id_hangar y dni y la política para borrado es restrict en ambas claves ya que no debemos permitir que se borre una persona física ni un hangar si aun tienen tuplas en la tabla alquila. Para actualización tenemos la política cascade ya que si queremos modificar el el dni o el id del hangar en su tabla correspondiente debemos hacerlo en la tabla alquila también para preservar la integridad.
- **Guarda:** tiene como claves foráneas id_hangar e id_avion y la política para borrado es restrict en ambas claves ya que no debemos permitir que se borre un avión ni un hangar si aun tienen tuplas en la tabla guarda. Para actualización tenemos la política cascade ya que si queremos modificar el el id del avión o el id del hangar en su tabla correspondiente debemos hacerlo en la tabla alquila también para preservar la integridad.
- **Realiza:** tiene como claves foráneas dni, id_avion y fecha para id_avion y fecha tenemos una política de cascade tanto para actualización como para borrado ya que deseamos que los cambios realizados en las tablas originales se propaguen en realiza. Para dni tenemos set null en borrado ya que nos interesa mantener las tuplas en realiza por mas que no tengan un mecánico asociado ya que el el avion puede seguir estando en la base de datos, para actualización usamos cascade.

b) Creación de tres dominios con diferentes restricciones

Dominios

- **domFechaNacimiento** lo aplicamos a las tablas Empleado y Piloto
Este dominio chequea que la persona sea mayor de edad con la siguiente comparativa:
 $\text{fechaActual} - \text{fechaNacimiento} > 6570 \text{ días (18 años expresados en días)}$
- **domDni** lo aplicamos a las tablas Mecanico, PersonaFisica y Piloto
Este dominio chequea que el DNI ingresado no sea nulo y en caso de no ingresarlo pone por defecto 99.999.999
- **domCuil** lo aplicamos a la tabla ObraSocial.
Este dominio chequea que el valor ingresado sea > 10000000000 (debe tener la cantidad minima de numeros que posee un CUIL)
- **domTipo** lo aplicamos a la tabla Mantenimiento
Este dominio chequea que el tipo de mantenimiento ingresado sea de uno de los valores permitidos de la siguiente manera:
domTipo check (value in (turbina, motor, fuselaje, tren de aterrizaje, cola, cabina, alas, helice)).

SQL DOMINIOS

```
CREATE DOMAIN domFechaNacimiento DATE  
CHECK ((CURRENT_DATE - VALUE) > 6570);
```

```
CREATE DOMAIN domDni INTEGER DEFAULT 99999999  
CHECK(value > 1000000 AND value < 99999999)
```

```
CREATE DOMAIN domCuil BIGINT  
CHECK (value > 10000000000);
```

```
CREATE DOMAIN domTipo VARCHAR(30)  
CHECK (value IN (  
    'turbina',  
    'motor',  
    'fuselaje',  
    'tren de aterrizaje',  
    'cola',  
    'cabina',  
    'alas',  
    'hélice'  
));
```

c) Definición de al menos un check basado en atributo

```
ALTER TABLE mecanico ADD COLUMN salario int CHECK(salario>0);
```

Este check basado en el atributo salario de mecánico compara que el número ingresado sea mayor a cero.

d) Definición de al menos un check basado en tuplas

```
ALTER TABLE alquiler ADD CONSTRAINT checkfechas CHECK (fecha_inicio < fecha_fin);
```

Este check basado en tupla compara que el atributo fecha_inicio de la relación alquiler sea menor a la fecha_fin de la misma relación

3) Realizar inserciones de todas las tablas. Al menos 5 tuplas en cada una de ellas.

Tuplas

Obra Social

nombre	ciudad	calle	altura	cuil
OSDE	Buenos Aires	Rivadavia	1200	20111111111
Swiss Medical	Córdoba	Colón	2300	20222222222
Galeno	Rosario	San Martín	540	20333333333
Medife	Mendoza	Las Heras	890	20444444444
IOMA	La Plata	7	1550	20555555555

Figura 2: Tuplas de Obra Social.

Mecánico

dni	nombre	apellido	ciudad	calle	turno	fecha_nacimiento	cuil	salario	altura
40000000	Juan	Pérez	Buenos Aires	San Martín	mañana	1985-04-12	20111111111	500000	12
40000001	Carlos	Lopez	Córdoba	Belgrano	tarde	1982-11-05	20222222222	800000	20
40000002	Pedro	Suarez	Rosario	Sarmiento	mañana	1990-07-23	20333333333	750000	25
40000003	Luis	Ramirez	Mendoza	San Juan	noche	1979-02-16	20444444444	1000000	20
40000004	Jorge	Fernandez	La Plata	Mitre	mañana	1992-09-30	20555555555	450000	15

Figura 3: Tuplas de Mecánico.

Piloto

dni	nombre	apellido	ciudad	calle	nro_licencia	fecha_nacimiento	cuil	altura
41000000	Mario	Gomez	Buenos Aires	San Martín	1001	1975-01-20	20111111111	12
41000004	Alberto	Diaz	La Plata	Mitre	1005	1985-08-27	20555555555	11
41000003	Matias	Sosa	Mendoza	San Juan	1004	1991-03-15	20444444444	23
41000002	Gabriel	Mendez	Rosario	Sarmiento	1003	1980-12-02	20333333333	546
41000001	Luciano	Vega	Córdoba	Belgrano	1002	1988-06-11	20222222222	56

Figura 4: Tuplas de Piloto.

Propietario

idpropietario	correo	ciudad	calle	altura
1	agusf399@gmail.com	neuquen	misiones	944
10	mili@gmail.com	cinco saltos	9 de julio	912
6	mati@gmail.com	centenario	buenos aires	800
2	avion@gmail.com	cinco saltos	san martin	150
3	1234@gmail.com	neuquen	roca	212
9	abc@gmail.com	cipolletti	mendoza	100
7	xyz@gmail.com	roca	san martin	777
5	jere@gmail.com	centenario	lainez	159
4	pedco@gmail.com	neuquen	mendoza	112
8	siu@gmail.com	neuquen	colon	674

Figura 5: Tuplas de Propietario.

Persona Fisica

dni	nombre	apellido	id_propietario
42000000	agustin	ferreyra	1
42000001	milagros	ruiz	2
42000002	nora	sanchez	3
42000004	jeremias	jaleniecki	4
42000005	matias	beroisa	5

Figura 6: Tuplas de Persona Fisica.

Corporación

nombre_unico	id_propietario
milaerolineas	10
pk	6
milaviones	7
river plate	9
jetsmart	8

Figura 7: Tuplas de Corporación.

Hangar

id_hangar	capacidad	ubicacion
1	3	Zona Norte
2	2	Zona Sur
3	5	Zona Este
4	4	Zona Oeste
5	3	Zona Central

Figura 8: Tuplas de Hangar.

Avión

id_avion	tipo	id_hangar	id_propietario	capacidad	peso	fecha_compra
1	Jet privado	1	1	12	7500	2018-03-15
2	Helicóptero	2	2	5	2300	2020-07-08
3	Avioneta	3	3	3	1800	2019-11-22
4	Carga ligera	1	4	2	9200	2021-05-13
5	Jet comercial	2	5	50	18000	2017-09-30

Figura 9: Tuplas de Avión.

Mantenimiento

fecha	id_avion	tipo	horas
2025-03-15	1	turbina	10
2025-07-08	2	motor	15
2025-06-22	3	fuselaje	20
2025-05-13	4	cola	6
2024-11-29	5	helice	8

Figura 10: Tuplas de Mantenimiento.

Realiza

dni	fecha	id_avion
40000000	2025-03-15	1
40000001	2025-07-08	2
40000002	2025-06-22	3
40000003	2025-05-13	4
40000004	2024-11-29	5

Figura 11: Tuplas de Realiza.

Alquila

id_hangar	dni	fecha_inicio	fecha_fin
1	42000000	2025-03-05	2025-04-01
5	42000005	2025-02-18	2025-02-25
4	42000004	2025-01-25	2025-02-10
3	42000002	2025-05-18	2025-06-02
2	42000001	2025-06-03	2025-06-15

Figura 12: Tuplas de Alquiler.

Teléfonos

id_propietario	telefono
1	2994615734
1	4569329201
2	5632345776
3	5432456787
3	5748392389
4	2654396538
4	1748365429
5	2748342325
6	5748122332
7	5748312646
8	4748675435
9	5748392389
10	274392389
10	274123389

Figura 13: Tuplas de Teléfonos.

Operaciones

4) Realizar una operación de borrado que involucre el borrado en cascada de alguna otra tabla. Tener en cuenta que la BD posee datos.

Borrado

Realizó la siguiente operación de borrado:

```
DELETE FROM propietario WHERE idpropietario = 2;
```

Las tablas afectadas fueron propietario, personaFisica, avion, teléfono, mantenimiento y realiza. A continuación se muestra el resultado en las tablas luego de realizar la operación:

Propietario

idpropietario	correo	ciudad	calle	altura
1	agusf399@gmail.com	neuquen	misiones	944
10	mili@gmail.com	cinco saltos	9 de julio	912
3	1234@gmail.com	neuquen	roca	212
9	abc@gmail.com	cipolletti	mendoza	100
7	xyz@gmail.com	roca	san martin	777
5	jere@gmail.com	centenario	lainez	159
4	pedco@gmail.com	neuquen	mendoza	112
8	siu@gmail.com	neuquen	colon	674
6	mati@gmail.com	centenario	buenos aires	800

Figura 14: No se encuentra el $id_propietario=2$.

Persona Fisica

dni	nombre	apellido	id_propietario
42000000	agustin	ferreyra	1
42000002	nora	sanchez	3
42000004	jeremias	jaleniecki	4
42000005	matias	beroisa	5
42000001	milagros	ruiz	NULL

Figura 15: $id_propietario$ es nulo en la última tupla.

Avión

id_avion	tipo	id_hangar	id_propietario	capacidad	peso	fecha_compra
1	Jet privado	1	1	12	7500	2018-03-15
3	Avioneta	3	3	3	1800	2019-11-22
4	Carga ligera	1	4	2	9200	2021-05-13
5	Jet comercial	2	5	50	18000	2017-09-30

Figura 16: El avión con $id_propietario = 2$ no se encuentra.

Teléfonos

id_propietario	telefono
1	2994615734
1	4569329201
3	5432456787
3	5748392389
4	2654396538
4	1748365429
5	2748342325
7	5748312646
8	4748675435
9	5748392389
10	274392389
10	274123389
6	5748122332

Figura 17: Los teléfonos con $id_propietario=2$ ya no se encuentran.

Mantenimiento

fecha	id_avion	tipo	horas
2025-03-15	1	turbina	10
2025-06-22	3	fuselaje	20
2025-05-13	4	cola	6
2024-11-29	5	helice	8

Figura 18: El mantenimiento con $id_avion = 2$ ya no se encuentra.

Realiza

dni	fecha	id_avion
40000000	2025-03-15	1
40000002	2025-06-22	3
40000003	2025-05-13	4
40000004	2024-11-29	5

Figura 19: La tupla con $id_avion = 2$ ya no se encuentra.

5) Realizar una operación de actualización que arroje error por definición de clave primaria y clave foránea. Debe mostrar la resolución con datos en las tablas que considere.

Error de actualización

Realizo la siguiente consulta:

```
UPDATE mecanico set dni = 44000000 where dni = 40000000;
```

que muestra el siguiente error:

```
Error al ejecutar consulta: ERROR: update or delete on table "mecanico" violates foreign key
constraint "realiza_dni_fkey" on table "realiza"
DETAIL: Key (dni)=(40000000) is still referenced from table "realiza".
```

Este error ocurre ya que se está modificando una clave primaria de una relacion que se encuentra como clave foránea en otra y tenemos una política de restrict al actualizar.

6) Definir cambios en los requerimientos del MER que cambien una relación a elección por una con atributos. Deben indicar el nuevo requerimiento en texto, mostrar en cambio en el MER, la derivación y por último realizar el SQL correspondiente al cambio.

Cambio de requerimientos del MER

Se desea registrar una fecha de ingreso y de salida de cada avión en un hangar determinado. Por lo tanto la relación GUARDA ahora posee los atributos fecha_ingreso y fecha_salida y la cardinalidad es ahora muchos a muchos ya que se permite que un avión se albergue en distintos hangares.

MER

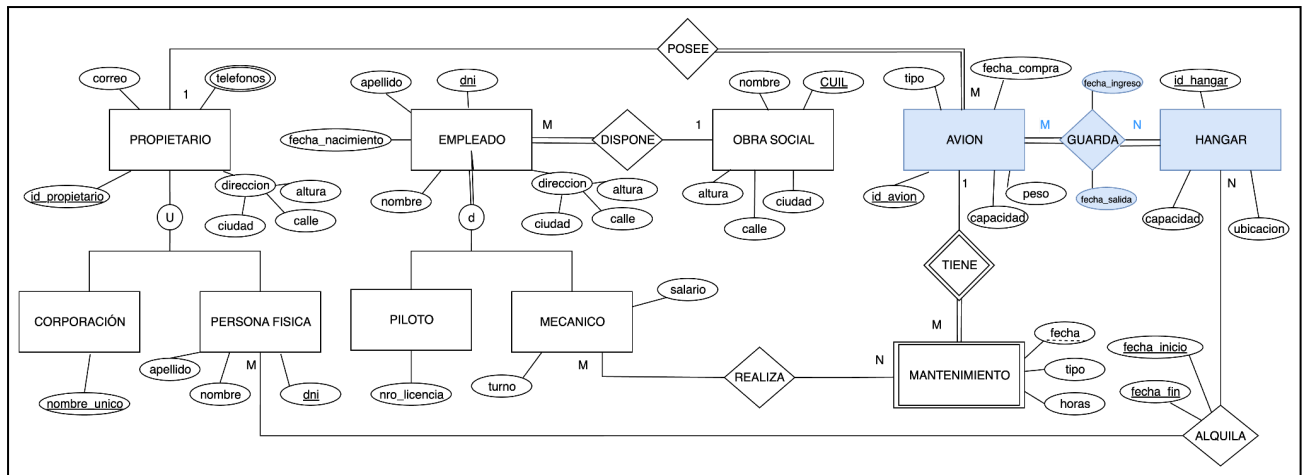


Figura 20: En azul se encuentran resaltados los cambios respecto al original

Derivación nueva de las tablas afectadas:

Avion(id_avion, tipo, capacidad, peso, fecha_compra, id_propietario)

Hangar(id_hangar, capacidad, ubicación)

Guarda(id_avion, id_hangar, fecha_ingreso, fecha_salida)

SQL de las tablas afectadas:

Guarda

<u>id_hangar</u>	<u>id_avion</u>	<u>fecha_ingreso</u>	<u>fecha_salida</u>
2	4	2025-03-11	2025-04-10
4	5	2025-04-15	2025-04-25
5	1	2025-02-01	2025-03-01
1	3	2025-06-15	2025-07-01
2	4	2025-02-01	2025-03-01
4	1	2025-08-01	2025-08-15

Figura 21: Tuplas de la relación guarda agregada

Avión

<u>id_avion</u>	tipo	<u>id_propietario</u>	capacidad	peso	fecha_compra
1	Jet privado	1	12	7500	2018-03-15
3	Avioneta	3	3	1800	2019-11-22
4	Carga ligera	4	2	9200	2021-05-13
5	Jet comercial	5	50	18000	2017-09-30

Figura 22: Tuplas de la relación Avión modificadas

Hangar

id_hangar	capacidad	ubicacion
1	3	Zona Norte
2	2	Zona Sur
3	5	Zona Este
4	4	Zona Oeste
5	3	Zona Central

Figura 23: Tuplas de la relación Hangar modificadas

7) Definir 3 afirmaciones que contengan (definir correctamente el requerimiento en lenguaje natural y luego el SQL):

Afirmaciones

a) Join entre dos tablas y un GROUP BY

Afirmación que controla que ningún hangar tenga más aviones almacenados que su capacidad

SQL:

```
CREATE ASSERTION capacidadAviones
CHECK(
    NOT EXIST(
        SELECT h.id_hangar
        FROM hangar h NATURAL JOIN guarda g
        GROUP BY h.id_hangar, h.capacidad
        HAVING count(g.id_avion) > h.capacidad
    )
)
```

b) Una función agregada de SQL y que posea un HAVING. Además debe utilizar NOT IN.

No se debe permitir que los mecanicos mayores a 50 años participen en más de 3 mantenimientos al día

SQL:

```
CREATE ASSERTION adultosMayores
CHECK(
    NOT EXIST(
```

```

SELECT m.dni
FROM mecánico m
WHERE EXTRACT(YEAR FROM age(m.fecha_nacimiento)) > 50
AND m.dni NOT IN(
    SELECT e.dni
    FROM mantenimiento e
    GROUP BY m.dni , m.fecha
    HAVING COUNT (e.fecha,e.id_avion) > 3
)
)
)

```

Corrección:

```

CREATE ASSERTION adultosMayores
CHECK(
    SELECT m.dni
    FROM mecánico m
    WHERE EXTRACT(YEAR FROM age(m.fecha_nacimiento)) > 50
    AND m.dni NOT IN(
        SELECT e.dni
        FROM mantenimiento e
        GROUP BY m.dni , m.fecha
        HAVING COUNT (e.fecha,e.id_avion) > 3
    )
)

```

- c) La restricción a cumplir debe ser comparando a un valor escalar como resultado de una función agregada.

Se desea asegurar que todos los salarios sean iguales o superiores al promedio de todos los salarios

SQL

```

CREATE ASSERTION adultosMayores
CHECK(
    ( SELECT MIN (salario) FROM mecanico ) >= (SELECT AVG(salario) FROM mecanico)
)

```

CORRECCIÓN:

Decidimos cambiar de afirmación:

Un propietario puede tener varios aviones, pero la suma del peso total de todos sus aviones no debe superar las 200 toneladas.

```
CREATE ASSERTION limite_peso_propietario CHECK (  
  NOT EXISTS (  
    SELECT id_propietario  
    FROM avion  
    GROUP BY id_propietario  
    HAVING SUM(peso) > 200000  
  )  
);
```

8) Definir 2 vistas que contengan (definir correctamente el requerimiento en lenguaje natural y luego el SQL):

Vistas

- a) 5 atributos en total (2 atributos de una tabla y 2 atributos de otra tabla y un valor resultante de una función agregada). Deben utilizar DATEDIFF y LIKE.

Creo una vista que muestra el id del hangar y su ubicación, el tipo de avión y su peso junto con el promedio de la cantidad de días que el avión estuvo en el hangar. Además filtrar los resultados para mostrar solo aquellos hangares cuya ubicación termina con la letra 'e'.

```
CREATE VIEW vista_aviones_hangar AS  
SELECT  
  h.id_hangar, h.ubicacion, a.tipo, a.peso, (CAST(g.fecha_salida AS date) - CAST(g.fecha_ingreso AS date)) AS  
  dias_en_hangar  
FROM guarda g JOIN hangar h ON g.id_hangar=h.id_hangar  
  
JOIN avion a ON g.id_avion = a.id_avion  
  
WHERE h.ubicacion LIKE '%e'  
  
GROUP BY h.id_hangar, h.ubicacion, a.tipo, a.peso
```

Corrección:

```
CREATE VIEW vista_aviones_hangar AS  
SELECT  
  h.id_hangar, h.ubicacion, a.tipo, a.peso,  
  AVG((CAST(g.fecha_salida AS date) - CAST(g.fecha_ingreso AS date))) AS  
  promedio_dias_en_hangar  
FROM guarda g  
  
  JOIN hangar h ON g.id_hangar = h.id_hangar
```

```

JOIN avion a ON g.id_avion = a.id_avion
WHERE h.ubicacion LIKE '%e'
GROUP BY h.id_hangar, h.ubicacion, a.tipo, a.peso;

```

vista_aviones_hangar

SELECT * FROM "vista_aviones_hangar" LIMIT 50 (0.001 s) Modificar						
☐ Modify	id_hangar	ubicacion	tipo	peso	dias_en_hangar	
☐ modificar	4	Zona Oeste	Jet privado	7500	14	
☐ modificar	4	Zona Oeste	Jet comercial	18000	10	
☐ modificar	1	Zona Norte	Avioneta	1800	16	

Figura 24: Tabla con los registros de la vista_aviones_hangar

- b) Utilizar Left Join para resolverla y al menos un INNER JOIN entre otras tablas.

Crea una vista que muestre: el avión, junto con su propietario, los teléfonos del propietario, y el hangar donde está guardado el avión.

```

CREATE VIEW vista_avion_propietario_hangar AS
SELECT
    a.id_avion, a.tipo, a.peso, p.idpropietario, p.correo, t.telefono, h.id_hangar, h.ubicacion
FROM avion a
INNER JOIN propietario p ON a.id_propietario = p.idpropietario
LEFT JOIN telefonos t ON p.idpropietario = t.id_propietario
INNER JOIN guarda g ON a.id_avion = g.id_avion
INNER JOIN hangar h ON g.id_hangar = h.id_hangar
ORDER BY a.id_avion;

```

vista_avion_hangar_propietario

SELECT * FROM "vista_avion_propietario_hangar" LIMIT 50 (0.001 s) Modificar								
☐ Modify	id_avion	tipo	peso	idpropietario	correo	telefono	id_hangar	ubicacion
☐ modificar	1	Jet privado	7500	1	agusf399@gmail.com	2994615734	4	Zona Oeste
☐ modificar	1	Jet privado	7500	1	agusf399@gmail.com	4569329201	4	Zona Oeste
☐ modificar	1	Jet privado	7500	1	agusf399@gmail.com	2994615734	5	Zona Central
☐ modificar	1	Jet privado	7500	1	agusf399@gmail.com	4569329201	5	Zona Central
☐ modificar	3	Avioneta	1800	3	1234@gmail.com	5432456787	1	Zona Norte
☐ modificar	3	Avioneta	1800	3	1234@gmail.com	5748392389	1	Zona Norte
☐ modificar	4	Carga ligera	9200	4	pedco@gmail.com	1748365429	2	Zona Sur
☐ modificar	4	Carga ligera	9200	4	pedco@gmail.com	2654396538	2	Zona Sur
☐ modificar	4	Carga ligera	9200	4	pedco@gmail.com	1748365429	2	Zona Sur
☐ modificar	4	Carga ligera	9200	4	pedco@gmail.com	2654396538	2	Zona Sur
☐ modificar	5	Jet comercial	18000	5	jere@gmail.com	2748342325	4	Zona Oeste

Figura 25 : Tabla con los registros de la vista _avion _propietario _hangar

Corrección: eliminamos los teléfonos del prop 3 para que se represente el leftjoin

☐ Modify	id_avion	tipo	peso	idpropietario	correo	telefono	id_hangar	ubicacion
☐ modificar	1	Jet privado	7500	1	agusf399@gmail.com	2994615734	4	Zona Oeste
☐ modificar	1	Jet privado	7500	1	agusf399@gmail.com	4569329201	4	Zona Oeste
☐ modificar	1	Jet privado	7500	1	agusf399@gmail.com	2994615734	5	Zona Central
☐ modificar	1	Jet privado	7500	1	agusf399@gmail.com	4569329201	5	Zona Central
☐ modificar	3	Avioneta	1800	3	1234@gmail.com	NULL	1	Zona Norte
☐ modificar	4	Carga ligera	9200	4	pedco@gmail.com	1748365429	2	Zona Sur
☐ modificar	4	Carga ligera	9200	4	pedco@gmail.com	2654396538	2	Zona Sur
☐ modificar	4	Carga ligera	9200	4	pedco@gmail.com	1748365429	2	Zona Sur
☐ modificar	4	Carga ligera	9200	4	pedco@gmail.com	2654396538	2	Zona Sur
☐ modificar	5	Jet comercial	18000	5	jere@gmail.com	2748342325	4	Zona Oeste

Unidad V - Seguridad

1. Dada la BD propuesta en el ejercicio definir:
 - a. 3 roles que posean:
 - i. Crear usuarios
 - ii. Conectarse a la BD
 - iii. Crear base de datos y pueda leer todas las tablas

Creación de roles

- Rol que permita crear usuarios:

Se crea el rol AdminHangar el cual tiene permisos para crear nuevos usuarios.

```
CREATE ROLE adminHangar WITH LOGIN CREATEROLE INHERIT PASSWORD 'admin123'
```

- Rol para conectarse a una base de datos:

Se crea el rol “trabajador” el cual podra conectarse a nuestra base de datos.

CREATE ROLE trabajador WITH LOGIN INHERIT PASSWORD 'trabajador123'

- Rol que permite crear bases de datos y leer en todas las tablas:
Se crea el rol “encargado” el cual tiene permiso de lectura en todas las tablas

CREATE ROLE encargado WITH LOGIN CREATEDB INHERIT PASSWORD 'encargado123'

para añadirle el permiso de lectura se ejecuta el siguiente comando:

GRANT SELECT ON ALL TABLES IN SCHEMA public TO encargado

- b. 6 usuarios que ...(al menos)
- 1 pueda leer solo un subconjunto de atributos de una tabla a elección
 - 2 usuarios que pertenezcan a uno de los roles del ejercicio 1.a
 - 1 usuario que pueda insertar solo a dos tablas a elección. Debe ser coherente si utiliza tablas derivadas de entidades débiles o especialización generalización o alguna restricción que se pueda derivar del modelo entidad relación. Entonces en lugar de dos pueden ser un par más.
 - 1 usuario que pueda contar con TODOS los privilegios
 - 1 usuario que pueda ejecutar una consulta SQL a los atributos de clave primaria y/o foránea de por lo menos 3 tablas. Debe escribir la consulta SQL.

Creacion de usuarios

- Usuario que pueda leer un subconjunto de atributos de una tabla:
Se crea el usuario: usuario_propietario que solo puede leer de la tabla avion los atributos: tipo, capacidad y fecha_compra. Esto se realiza ya que el resto de atributos son o bien privados o bien no aportan informacion útil para el propietario de un avion.
La figura 26 muestra todos los atributos de la tabla Avión

Avión

Columna	Tipo
id_avion	integer
tipo	character varying(20) NULL
id_hangar	integer NULL
id_propietario	integer NULL
capacidad	smallint
peso	smallint
fecha_compra	date

Figura 26: Tabla con los atributos de la tabla Avión


```
CREATE USER usuario_propietario WITH PASSWORD 'propietario123'
```

Para darle permisos de lectura de los atributos requeridos se inserta el siguiente comando:

```
GRANT SELECT (fecha_compra, tipo, capacidad) ON TABLE avion TO usuario_propietario
```

- Usuarios pertenecientes a un rol creado previamente:

Se crea el usuario: usuario_adminHangar y se le asigna el rol de “AdminHangar”

```
CREATE USER usuario_adminHangar WITH LOGIN ENCRYPTED PASSWORD 'admin123' IN ROLE adminHangar
```

Se crea el usuario: usuario_encargado y se le asigna el rol de “encargado”

```
CREATE USER usuario_encargado WITH LOGIN ENCRYPTED PASSWORD 'encargado123' IN ROLE encargado
```

- Usuario que puede insertar a dos tablas a elección:

Se crea el usuario: usuario_trabajador el cual tiene permitido insertar en las tablas Mantenimiento y Realiza ya que un trabajador es el encargado de realizar los mantenimientos de un avión y por lo tanto necesita poder cargarlos en la base de datos.

```
CREATE USER usuario_trabajador WITH LOGIN ENCRYPTED PASSWORD 'trabajador123'
```

```
GRANT INSERT ON TABLE realiza TO usuario_trabajador
```

```
GRANT INSERT ON TABLE mantenimiento TO usuario_trabajador
```

- Usuario que cuente con todos los privilegios

Se crea el usuario “superAdmin” el cual posee todos los privilegios para gestionar el aeropuerto.

```
CREATE USER superAdmin WITH LOGIN ENCRYPTED PASSWORD 'superadmin'
```

```
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO superAdmin
```

- Usuario que pueda ejecutar una consulta SQL a los atributos clave primaria y/o foránea de por lo menos 3 tablas:

Se crea un usuario “usuario_organizador” el cual tiene como objetivo acceder a las claves primarias y foraneas de la tabla, avión, propietario y hangar. El objetivo de este usuario es poder ver los aviones, donde se encuentran almacenados y sus respectivos dueños de una manera rápida para facilitar la organización de los hangares. Creamos 3 vistas auxiliares vista_clavesAvion, vista_clavesPropietario y vista_clavesHangar

Creación del usuario:

```
CREATE USER usuario_organizador WITH LOGIN ENCRYPTED PASSWORD 'organizador123'
```

Creación de las vistas:

```
CREATE VIEW vista_clavesAvion AS
```

```
SELECT id_avion, id_propietario
```

```
FROM avion
```

```
CREATE VIEW vista_clavesHangar AS
```

```
SELECT id_hangar
```

```
FROM hangar
```

```
CREATE VIEW vista_clavesPropietario AS
```

```
SELECT idpropietario
```

```
FROM propietario
```

Asignación de las vistas al usuario:

```
GRANT SELECT ON vista_clavesAvion TO usuario_organizador
```

```
GRANT SELECT ON vista_clavesPropietario TO usuario_organizador
```

```
GRANT SELECT ON vista_clavesHangar TO usuario_organizador
```

- Realice en forma escrita una tabla de usuarios/recursos/permisos en donde se pueda visualizar el resultado de las sentencias definidas en el ejercicio anterior.

Tabla de permisos de usuarios

La tabla 1 indica los permisos que posee cada usuario(columna) respecto de cada relación (fila). Indice: N/A(not allowed) ALL(todos los permisos) Select(solo permisos de lectura) INSERT(permiso de escritura)

	usuario_ propietario	usuari o_ admin Hanga r	usuario _ encarga do	usuari o_ trabaj ador	usuari o_ organi zador	super Admin	admin Hangar	trabajad or	encargado
avión	SELECT(tipo, capacidad,fec ha_ compra)	ALL	SELEC T	N/A	N/A	ALL	ALL	SELECT	SELECT

hangar	N/A	ALL	SELEC T	N/A	N/A	ALL	ALL	SELECT	SELECT
mecánico	N/A	ALL	SELEC T	N/A	N/A	ALL	ALL	SELECT	SELECT
piloto	N/A	ALL	SELEC T	N/A	N/A	ALL	ALL	SELECT	SELECT
propieta rio	N/A	ALL	SELEC T	N/A	N/A	ALL	ALL	N/A	SELECT
persona física	N/A	ALL	SELEC T	N/A	N/A	ALL	ALL	N/A	SELECT
corporac ión	N/A	ALL	SELEC T	N/A	N/A	ALL	ALL	N/A	SELECT
manteni miento	N/A	ALL	SELEC T	INSER T	N/A	ALL	ALL	SELECT	SELECT
alquila	N/A	ALL	SELEC T	N/A	N/A	ALL	ALL	N/A	SELECT
realiza	N/A	ALL	SELEC T	INSER T	N/A	ALL	ALL	SELECT	SELECT
teléfonos	N/A	ALL	SELEC T	N/A	N/A	ALL	ALL	N/A	SELECT
obra social	N/A	ALL	SELEC T	N/A	N/A	ALL	ALL	N/A	SELECT
vista_ clavesPro pietario	N/A	ALL	SELEC T	N/A	SELE CT	ALL	ALL	N/A	SELECT
vista_ clavesAvi on	N/A	ALL	SELEC T	N/A	SELE CT	ALL	ALL	N/A	SELECT
vista_ clavesHa ngar	N/A	ALL	SELEC T	N/A	SELE CT	ALL	ALL	N/A	SELECT

Tabla 1 : Tabla de permisos de usuario

Unidad V - Recuperación

- Realizar una bitácora de una planificación que posea 4 transacciones que se hayan ejecutado o están ejecutándose. Se deben considerar al menos 25 operaciones (distribuidas entre las transacciones) y luego un fallo. La bitácora debe cumplir que, luego del fallo:
 - Cuando el mecanismo de recuperación sea inmediato, deben haber transacciones tanto en la lista de deshacer como en la lista de rehacer.
 - Cuando el mecanismo de recuperación sea diferido, deben haber transacciones en la lista de rehacer.
- Agregar checkpoints a la bitácora anterior que reflejen cambios en las listas generadas en los puntos 1) a y b.
- Demostrar con una traza lo solicitado en cada uno de los ejercicios anteriores.

Transacciones con fallo

Los datos que utilizaremos son A,B,C,D,E.

Los valores iniciales son: A=0, B=200, C=30, D=70, E=50.

T1	T2	T3	T4
	READ(A)		
	READ(B)		
		READ(C)	
		C=C+20	
		WRITE(C)	
	B= B-100		
			READ(D)
	WRITE(B)		
READ(E)			
			D= D+100
			WRITE(D)
E=E-20			
WRITE(E)			
FALLO			
	A=A+40		
	WRITE(A)		

Tabla 2 : Tabla de transacciones con fallo

Mecanismo de recuperación inmediato

Las transacciones se ejecutan en el siguiente orden: T2,T3,T4,T1

BITÁCORA	BASE DE DATOS
<T2,begin>	A=0 B=200 C=30 D=70 E=50
<T3,begin>	
<T3,C,30,50>	A=0 B=200 C=50 D=70 E=50
<T3,commit>	
<T4,begin>	
<T2,B,200,100>	A=0 B=100 C=30 D=70 E=50
<T1,begin>	
<T4,D,70,170>	A=0 B=200 C=30 D=170 E=50
<T4,commit>	
<T1,E,50,30>	A=0 B=200 C=30 D=70 E=30

Tabla 3 : Traza de mecanismo de recuperación inmediata

Luego de realizar la traza se deben realizar las siguientes acciones:

Deshacer<T1,T2>

Rehacer<T3,T4>

Mecanismo de recuperación diferido

Las transacciones se ejecutan en el siguiente orden: T2,T3,T4,T1

BITÁCORA	BASE DE DATOS
<T2,begin>	A=0 B=200 C=30 D=70 E=50
<T3,begin>	
<T3,C,50>	
<T3,commit>	A=0 B=200 C=50 D=70 E=50
<T4,begin>	
<T2,B,100>	
<T1,begin>	
<T4,D,170>	
<T4,commit>	A=0 B=100 C=30 D=170 E=50
<T1,E,30>	

Figura 30 : Traza con mecanismo de recuperación diferido

Luego de realizar la traza se deben realizar las siguientes acciones:

Rehacer<T3,T4>

Mecanismo de recuperación inmediato con checkpoint

Los checkpoints se realizan cada 1 minuto y 30 segundos.

Las transacciones se ejecutan en el siguiente orden: T2,T3,T4,T1.

TIEMPO	BITÁCORA	BASE DE DATOS
10 segundos	<T2,begin>	A=0 B=200 C=30 D=70 E=50
20 segundos	<T3,begin>	
30 segundos	<T3,C,30,50>	A=0 B=200 C=50 D=70 E=50
40 segundos	<T3,commit>	
50 segundos	<T4,begin>	
1 minuto	<T2,B,200,100>	A=0 B=100 C=30 D=70 E=50
1 minuto 10 segundos	<T1,begin>	
1 minuto 20 segundos	<T4,D,70,170>	A=0 B=200 C=30 D=170 E=50
1 minuto 30 segundos	<checkpoint>	
1 minuto 40 segundos	<T4,commit>	
1 minuto 50 segundos	<T1,E,50,30>	A=0 B=200 C=30 D=70 E=30

Figura 31: Traza con mecanismo de recuperación inmediato con checkpoint

Luego de realizar la traza se deben realizar las siguientes acciones:

Deshacer<T1,T2>

Rehacer<T4>

Mecanismo de recuperación diferido con checkpoint

Los checkpoints se realizan cada 1 minuto y 30 segundos.

Las transacciones se ejecutan en el siguiente orden: T2,T3,T4,T1.

TIEMPO	BITÁCORA	BASE DE DATOS
10 segundos	<T2,begin>	A=0 B=200 C=30 D=70 E=50
20 segundos	<T3,begin>	
30 segundos	<T3,C,50>	
40 segundos	<T3,commit>	A=0 B=200 C=50 D=70 E=50
50 segundos	<T4,begin>	
1 minuto	<T2,B,100>	
1 minuto 10 segundos	<T1,begin>	
1 minuto 20 segundos	<checkpoint>	
1 minuto 30 segundos	<T4,D,170>	
1 minuto 40 segundos	<T4,commit>	A=0 B=100 C=30 D=170 E=50
1 minuto 50 segundos	<T1,E,30>	

Figura 32: Traza con mecanismo de recuperación diferido con checkpoint

Luego de realizar la traza se deben realizar las siguientes acciones:

Rehacer<T4>

Participaciones

Grupal

Seleccionamos el dominio y realizamos el MER correspondiente. Luego creamos las tablas en SQL, cargamos las tuplas y definimos los dominios.

Ruiz Milagros

Se encargó de aplicar la integridad referencial en SQL. Además realizó las modificaciones pedidas al dominio en la unidad I ejercicio 6, las afirmaciones y las vistas solicitadas.

Ferreya Agustín

Se encargó de crear los roles, usuarios y la tabla de permisos. Realizó la bitácora de planificación y las trazas para los distintos mecanismos de recuperación ante fallos.